

Python and computational research environments

What Is Python?

- Python is a programming language.
- Just like we use language to give instructions to people, we can use a programming language to give instructions to a computer.

For example:

- **You want:** Find the average of 1,000 survey responses.
- **Python:** A language telling the computer how to do it.
- **Computer:** Runs the instructions and gives you the result

Why Python in a Research Methods Class?

- Researchers often work with survey data, interview transcripts, news articles, social media posts
- Sometimes we have **too much information to work with manually**. Python can help us work with these data using a computer.

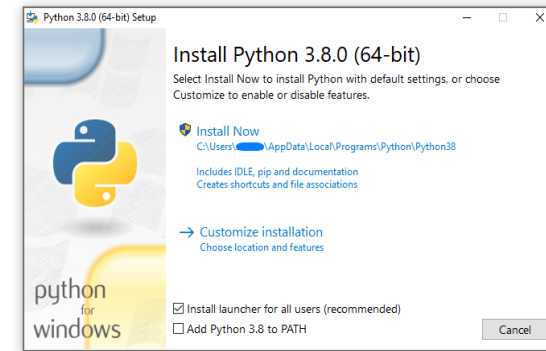
For example:

- **10 interviews?** → You might read them yourself.
- **100,000 social media posts?** → A computer can help.

Other reasons to use Python

- Free and open source
- Supports text and numeric data analysis
- Support AI-assisted research process (e.g., literature review, text data collection, text data analysis)

Where Can We Use Python?



What Is Google Colab?

- Google Colab is a website where we can write and run Python.
- Python $\neq$ Colab
- Python = the language
Colab = one place where we use it

An Easy Way to Think About It

- **English** = language
 Microsoft Word = place where you can write English
- Similarly:
- **Python** = programming language
 Google Colab = place where you can write and run Python



Why We Will Use Google Colab



- Runs in a web browser
 - No local installation
 - Easy sharing
 - Works across operating systems
 - Interactive notebooks
-

What Do you Need to Know Before Using CoLab

- **Runtime:** the active computing session
- **CPU:** standard option for data cleaning, and statistics
- **GPU:** useful for heavier neural-network or Transformer tasks
- **TPU:** advanced option for large machine-learning workloads

Now, open your Colab

Anatomy of Google Colab

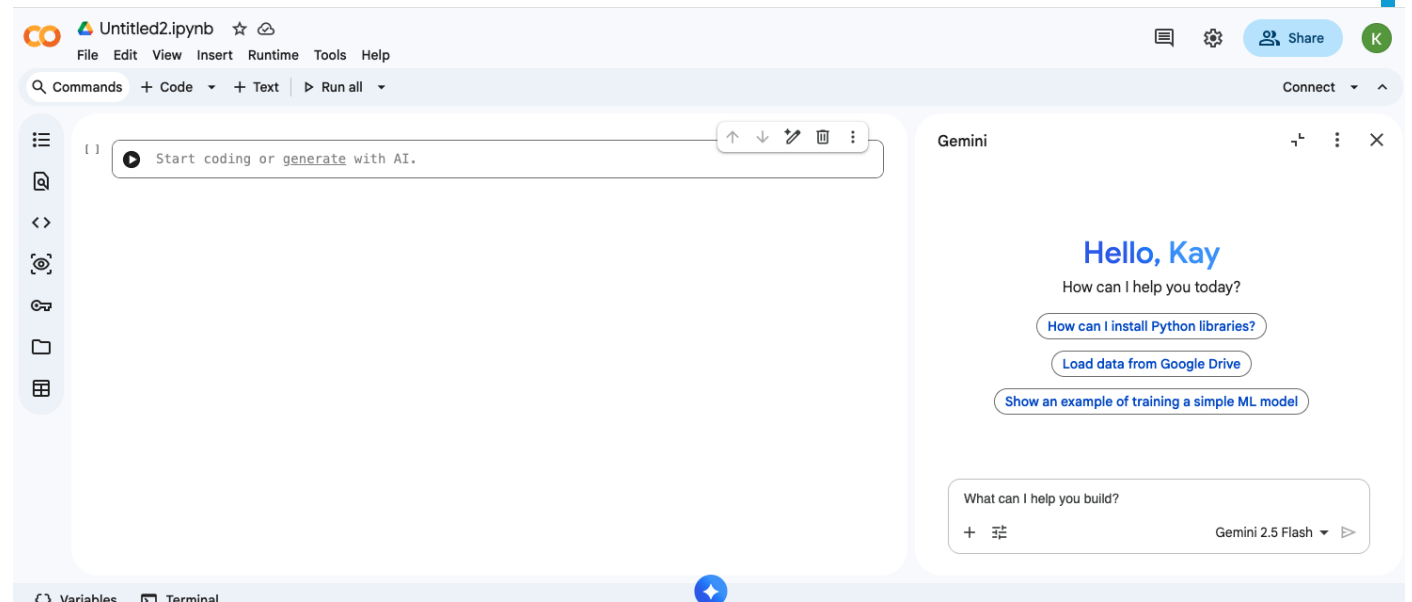
The screenshot displays the Google Colab web interface. At the top, the header includes the Colab logo, the file name "Untitled2.ipynb", and a menu bar with options: File, Edit, View, Insert, Runtime, Tools, and Help. To the right of the menu are icons for chat, settings, a "Share" button, and a user profile icon labeled "K". Below the header is a "Commands" search bar and tabs for "+ Code", "+ Text", and "Run all".

The main workspace is divided into two panels. The left panel is the code editor, showing a single code cell with the text "Start coding or generate with AI." and a toolbar with icons for undo, redo, insert, delete, and a menu. The right panel is the Gemini AI chat interface, titled "Gemini". It displays a greeting "Hello, Kay" and the question "How can I help you today?". Below this are three suggested prompts in blue buttons: "How can I install Python libraries?", "Load data from Google Drive", and "Show an example of training a simple ML model". At the bottom of the chat panel is a text input field with the placeholder "What can I help you build?", a plus icon, a Gemini logo, and a dropdown menu set to "Gemini 2.5 Flash" with a send button.

At the bottom of the interface, there are tabs for "Variables" and "Terminal", and a blue circular icon with a white star in the center.

What Is a Notebook?

- Code cells
- Text cells
- Outputs
- Tables
- Figures
- Explanations and analysis together



How Google Colab Works

- Notebook saved as an .ipynb file
- Code runs on a temporary Google virtual machine
- Standard CPU is enough for most course tasks
- GPU or TPU access is optional and limited
- Runtime variables and uploaded files may disappear
- Save notebooks, data, and outputs separately
- DO NOT upload any sensitive information

Python Is Literal: It Does Exactly What You Tell It to Do

- Executes exactly what we write
- Small changes can change results
- Some mistakes produce errors
- Other mistakes produce wrong answers

First Steps Python as a Calculator

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /
- Powers: **
- Parentheses control order



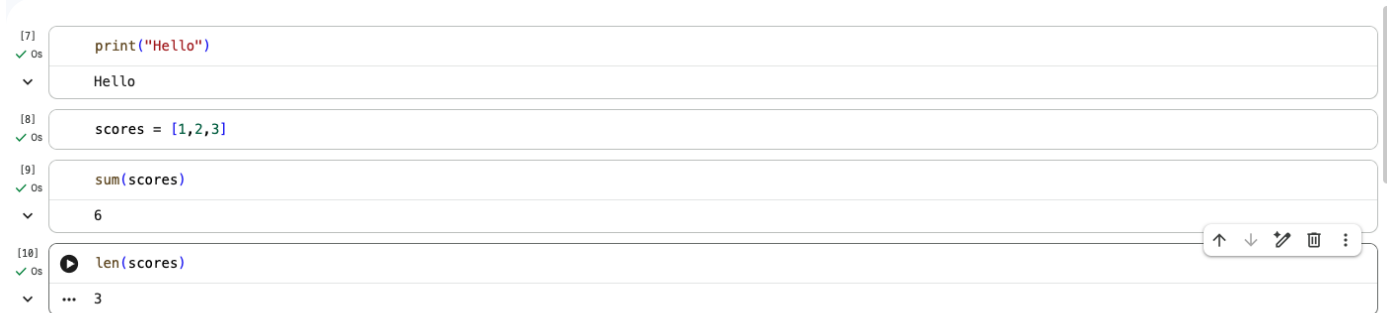
Reading Python Code

Symbol	Think of it as...	Simple example
()	Do something / give something information	<code>print("Hello")</code>
[]	A group of things OR pick something from a group	<code>names[0]</code>
.	Use something that belongs to something else	<code>text.upper()</code>
" " or ' '	This is text	<code>"Hello"</code>
,	Separate things	<code>print("Kay", 10)</code>
#	A note for humans — Python ignores it	<code># My analysis</code>
=	Give something a value	<code>age = 25</code>
==	Ask: Are these equal?	<code>age == 25</code>

Now that you know the basics of how to read Python code, let's learn about one of the most useful building blocks in Python: functions

Functions perform actions. When you write a function and want it to perform a specific action:

- Start with the function name
- Then parentheses
- Provide input or argument
- You will then get the returned output



```
[7]
✓ Os print("Hello")
Hello

[8]
✓ Os scores = [1,2,3]

[9]
✓ Os sum(scores)
6

[10]
✓ Os len(scores)
... 3
```

The screenshot shows a Jupyter Notebook interface with three code cells. The first cell contains `print("Hello")` and outputs `Hello`. The second cell contains `scores = [1,2,3]` and has no visible output. The third cell contains `sum(scores)` and outputs `6`. The fourth cell contains `len(scores)` and outputs `3`. Each cell is preceded by a prompt number in brackets (e.g., [7], [8], [9], [10]) and a status icon (a green checkmark and 'Os'). The output of the last cell is truncated with an ellipsis and a scroll bar is visible on the right side of the notebook area.

The next useful building block is variables

- Variable store Information
- In other words, it's how we give information a name
- Use = to assign a value
- We can reuse these values in later steps



```
[1] participants = 120
    response_rate = 0.75

[2] responses = participants * response_rate

[3] responses
... 90.0
```

Basic Data Types

- Integer: 120
- Float: 0.75
- String: "communication research"
- Boolean: True or False
- List: [72, 81, 90]
- Dictionary (mapping key and values: {"name": "Alex", "score": 90})

Checking Data Types

- Use `type()`
- This is to confirm how Python reads a value
- Important before analysis

Strings as Research Data

- Interview responses
- News articles
- Social-media posts
- Survey comments
- Model prompts
- Model outputs



Numbers and Strings (i.e., Text) Behave Differently



Basic String Operations

- Capitalize: `text.upper()`
- Lowercase: `text.lower()`
- Split: `text.split()`
- Replace: `text.replace()`
- Count characters: `len(text)`
- Combine: `text1 + text2`

What Are Python Libraries?

- **Think about Excel:**
- Excel already has tools for things like:
- calculating averages
- making charts
- organizing data
- Python can do these things too, but many of its tools come in *libraries*.

A Library = Extra Tools We Can Use in Python

- **pandas** → work with data
- **matplotlib** → make graphs
- **transformers** → work with language models
- Think: Python = your workspace | Libraries = toolboxes you bring into the workspace



```
import statistics
statistics.mean(scores)
```

... 2



Libraries We Will Use

- pandas — data tables
- Scipy, pingouin, or statsmodels — statistics
- requests — APIs
- transformers — language models
- matplotlib — visualization

Comments Explain Code

- Begin with #
- Ignored by Python
- Explain decisions
- Help future readers

Python for Simple Statistics

- Count: `len(scores)`
- Total: `sum(scores)`
- Minimum: `min(scores)`
- Maximum: `max(scores)`
- Mean: `statistics.mean(scores)`
- Median: `statistics.median(scores)`
- Standard deviation: `statistics.stdev(scores)`

```
[22]
✓ Os
import pandas as pd
df = pd.DataFrame({'col1': [1, 2], 'col2': [3, 4]})
df
```

	col1	col2
0	1	3
1	2	4

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
[23]
✓ Os
df.mean()
df.median()
df.std()
df.min()
df.max()
df.count()
df.value_counts()
df.describe()
```

	col1	col2
count	2.000000	2.000000
mean	1.500000	3.500000
std	0.707107	0.707107
min	1.000000	3.000000
25%	1.250000	3.250000
50%	1.500000	3.500000
75%	1.750000	3.750000
max	2.000000	4.000000

Errors Are Part of Programming

- Read the final line
- Identify the error type
- Check spelling
- Check quotation marks
- Check parentheses
- Rerun the corrected cell

AI Can Help With Code

- Explain unfamiliar code
- Interpret error messages
- Suggest corrections
- Generate starting examples
- Verify before using
- Understand before submitting

Research Workflow in the Age of AI

- Write an instruction
- Run the code
- Inspect the output
- Check for errors
- Revise the instruction
- Document the process

Next: More about Python fundamentals in Colab

- More terms related to research data such as lists and dictionaries
- How to use Python, Colab, and LLMs with literature review